Programação Orientada a Objetos

Prof. Perini

Aula 6 – Encapsulamento

Encapsulamento

- É um recurso que protege os dados da classe, alterando a sua visibilidade, porém garantindo o acesso a eles por meio de métodos especiais: get e set.

- O encapsulamento é um mecanismo que permite o acesso aos dados de um objeto somente através dos métodos desse. Nenhuma outra parte do programa pode operar sobre os dados do objeto. A comunicação entre os objetos é feita apenas através de troca de mensagens. (ENADE-2011)

Modificadores de Acesso

- public: significa visibilidade total, ou seja, o atributo pode ser acessado dentro da própria classe e em qualquer outra classe, de qualquer pacote.

- private: o elemento tem visibilidade somente dentro de sua própria classe. Outras classes não podem acessá-lo diretamente.

- protected: permite que o atributo ou método seja visualizado na própria classe e em suas classes filhas (veremos em herança).

- default: sem modificador de visibilidade. Permite o acesso das classes que estão no mesmo pacote.

Encapsulamento

- A visibilidade dos atributos define a forma com que os atributos estarão disponíveis.

```
// Atributos (características do carro)
private String modelo;
private String cor;
private String motor;
private int velocidadeAtual;
```

Encapsulamento – getters e setters

```
// getters e setters:
public String getModelo() {
    return modelo;
}
public void setModelo(String modelo) {
    this.modelo = modelo;
}
public String getCor() {
    return cor;
}
public void setCor(String cor) {
    this.cor = cor;
}
public String getMotor() {
    return motor;
}
public void setMotor(String motor) {
    this.motor = motor;
}
public int getVelocidadeAtual() {
    return velocidadeAtual;
}
public void setVelocidadeAtual(int velocidadeAtual) {
    this.velocidadeAtual = velocidadeAtual;
}
```

Encapsulamento – toString

Retorna os valores do objeto numa String.

```
// toString:
public String toString() {
    return "Carro modelo: " + modelo + "\n"+
            "Cor: "+ cor + "\n"+
            "Motor: " + motor;
}
```

Construtores

- A instrução new é utilizada na criação de objetos, que são instâncias de uma classe.
- Na orientação a objetos, é declarado um método com o mesmo nome da classe, chamado método construtor.
- O construtor inicia o objeto e pode fazer isso de duas formas:
  - Forma padrão: Cria o objeto com seus atributos numéricos valendo zero e strings como null.
  - Forma sobrecarregada: já inicia configurando os valores de seus atributos com os números e strings digitados pelo usuário.

Construtores

```
// Construtor: Um método especial chamado ao criar um novo objeto.
// Ele inicializa os atributos do carro.
public Carro() {   // construtor padrão

}
public Carro(String modeloVal, String corVal, String motorVal) {  // forma sobrecarregada
    modelo = modeloVal;
    cor = corVal;
    motor = motorVal;
    velocidadeAtual = 0; // Carro começa parado
}
```

Instanciando dois carros

```
Carro carro1 = new Carro("Sedan", "Preto", "1.6");

Carro carro2 = new Carro();
carro2.setModelo("Fiesta");
carro2.setCor("Prata");
carro2.setMotor("1.0");
```

Exercícios Práticos

Exercício 1: Sistema de Gestão de Contas Bancárias

Objetivo: Criar uma abstração para uma conta bancária onde o saldo não pode ser alterado diretamente, apenas através de operações de depósito e saque.

1. Classe Modelo (ContaBancaria.java):
   - Atributos privados: numeroConta (int), titular (String) e saldo (double).
   - Construtor: Deve inicializar o número e o titular. O saldo começa em 0.
   - Encapsulamento: Crie apenas o get para o saldo (não deve haver setSaldo, pois o saldo só muda via operações). Crie get/set para o número da conta e o titular.
   - Crie o toString para retornar todos os valores da conta.
   - Métodos: depositar(double valor) e sacar(double valor). O saque só deve ocorrer se houver saldo suficiente.
2. Classe de Execução (BancoApp.java):
   - No método main, instancie uma conta.
   - Tente alterar o saldo diretamente (e observe o erro de compilação).
   - Realize um depósito de R$ 500,00 e um saque de R$ 200,00, exibindo o saldo após cada operação.

Exercícios Práticos

Exercício 2: Sistema de Controle de Estoque de Produtos

Objetivo: Controlar produtos em um estoque, garantindo que o preço e a quantidade nunca sejam valores negativos.

1. Classe Modelo (Produto.java):
   - Atributos privados: nome (String), preco (double) e quantidadeEstoque (int).
   - Construtor: Inicializa todos os campos.
   - Encapsulamento (Lógica de Validação):
     - No setPreco(double novoPreco), só aceite o valor se for maior que zero.
     - No setQuantidadeEstoque(int novaQtd), só aceite se o valor for maior ou igual a zero.
     - Crie os métodos get correspondentes.
     - Crie o toString para retornar todos os valores do produto.
2. Classe de Execução (EstoqueApp.java):
   - Instancie um produto (ex: "Teclado Mecânico", R$ 250.00, 15 unidades).
   - Tente definir um preço negativo via setPreco e verifique se a sua lógica impediu a alteração.
   - Exiba os dados finais do produto formatados.